



中国科学技术大学
University of Science and Technology of China

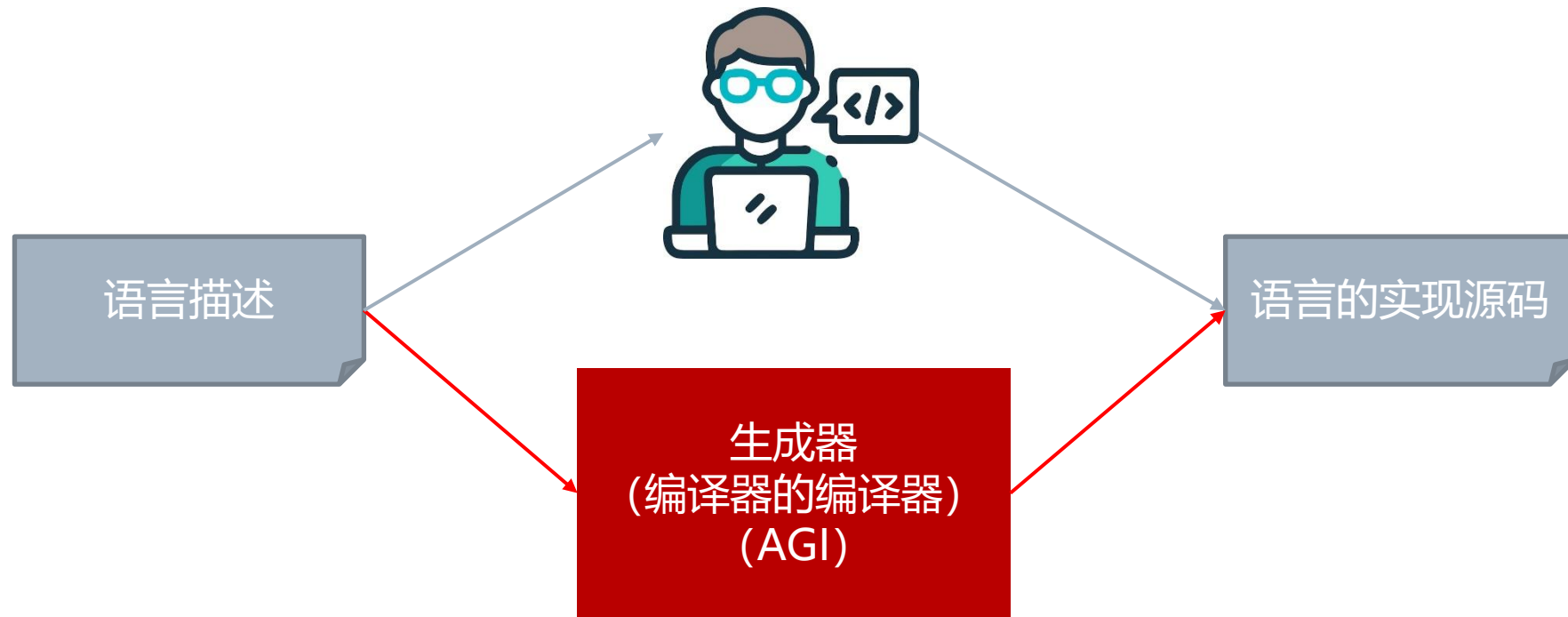
语言的解析 – 自动生成

《编译原理和技术(H)》

张昱

yuzhang@ustc.edu.cn

中国科学技术大学
计算机科学与技术学院





分析器的生成器

□ 生成器

- 生成Lexer: [Flex](#) ([for windows](#))、[Jflex](#)
- 生成Parser
 - LALR: [Bison](#) ([for windows](#))、[Java CUP](#)
 - LL: [JavaCC](#)、[ANTLR](#) – LL(*)^[PLDI2011], AdaptiveLL(*)^[OOPSLA2014]

□ 文法对Parser的影响

- 自下而上Parser的优势: 速度快、表达能力强
- 自上而下Parser的优势:
代码结构与文法对应, 易理解, 容易增加错误处理和错误恢复



中国科学技术大学
University of Science and Technology of China

词法分析



用Lex建立词法分析器

词法分析器 — Lexical analyzer, scanner
生成器 — generator

Flex <https://github.com/westes/flex> ; **JFlex** <http://jflex.de/>

ANTLR <https://www.antlr.org/> v4.8 Jan 16, 2020 v4.9.2 Mar 11, 2021 v4.11.1 Sept 4, 2022

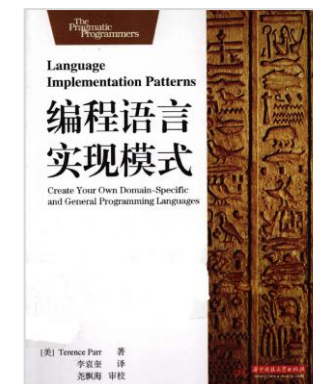
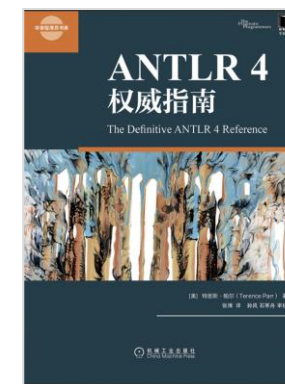
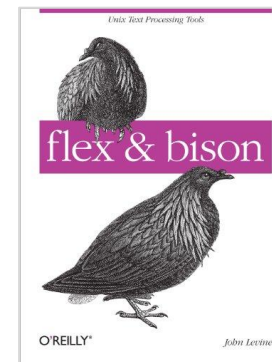
v4.13.2 Aug. 2024

<http://lab.antlr.org/>



Terence Parr is a **tech lead at Google** and until 2022 was a professor of **data science / computer science** at **Univ. of San Francisco**. He is the maniac behind ANTLR and has been working on language tools since 1989.

Check out Terence impersonating a machine learning droid: explained.ai



- [Java](#)
- [C#](#) (and an [alternate C# target](#))
- [Python](#) (2 and 3)
- [JavaScript](#)
- [TypeScript](#)

- [Go](#)
- [C++](#)
- [Swift](#)
- [PHP](#)
- [DART](#)



用Lex建立词法分析器

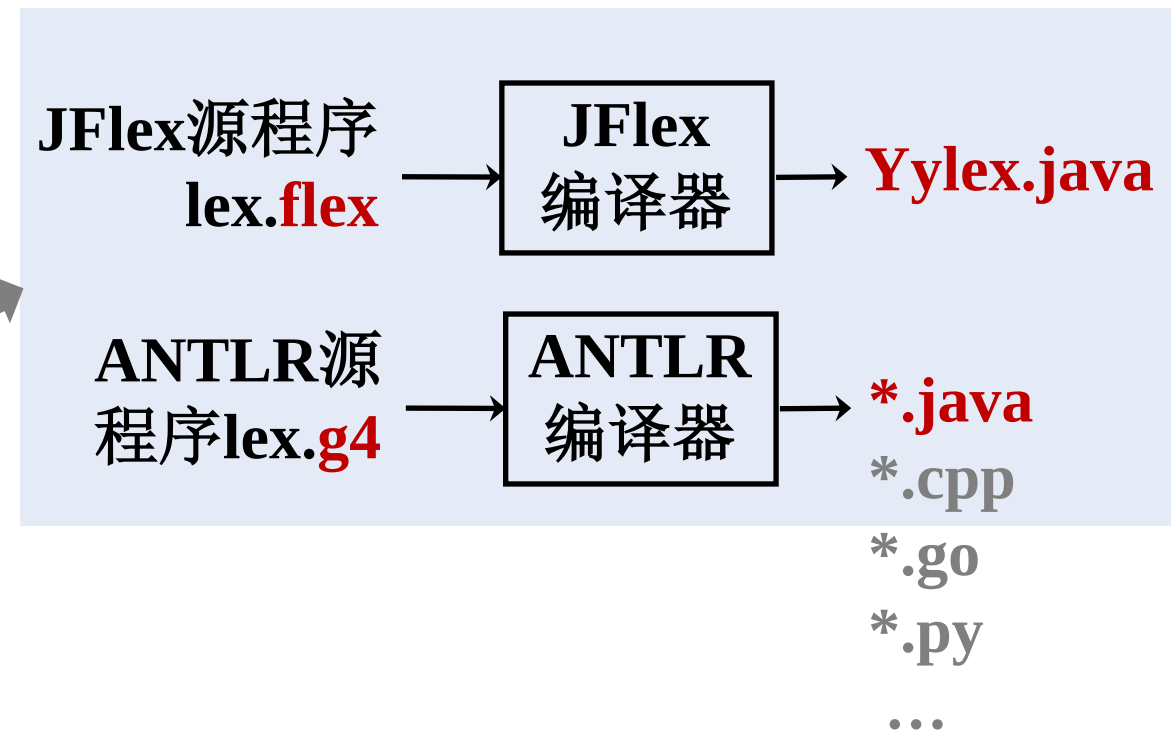
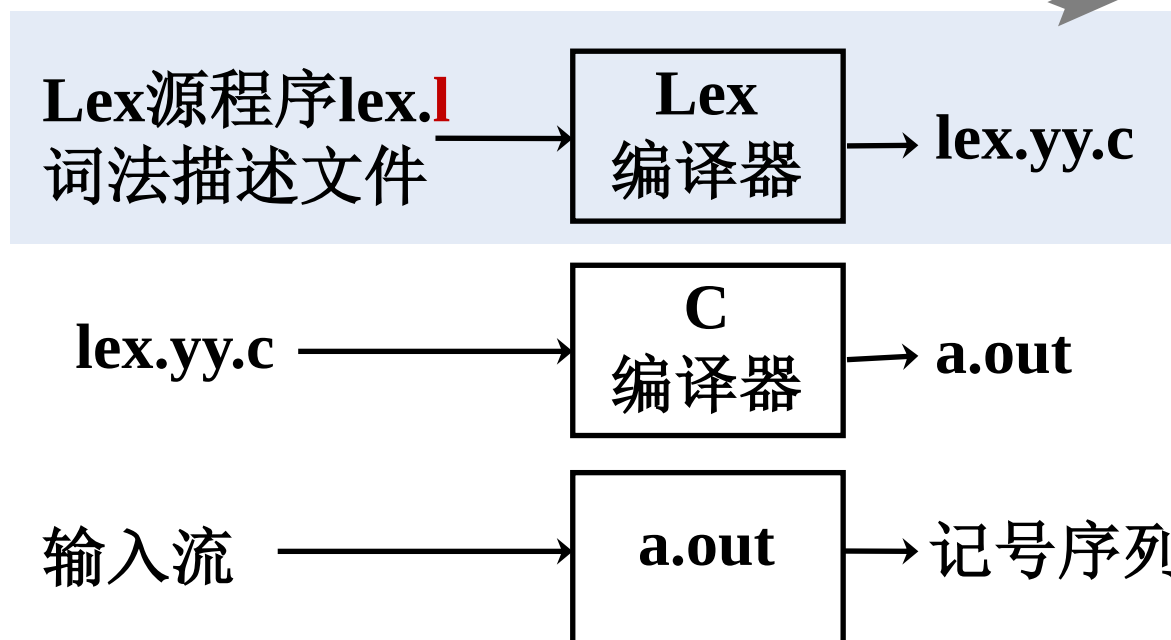
词法分析器 — Lexical analyzer, Scanner

生成器 — Generator

Flex <https://github.com/westes/flex>

JFlex <http://jflex.de/>

ANTLR <https://www.antlr.org/>





关联的实验资源

□ 工具及C++高级特征理解

- 1 GCC 与 Clang 工具链使用: [Educoder](#)、[校内](#) ([IP链接](#))
- 2 理解和使用 C++ 运行时类型信息 (RTTI) : [Educoder](#)、[校内](#) ([IP链接](#))
- 3 理解和使用 C++ 的智能指针: [Educoder](#)、[校内](#) ([IP链接](#))

□ 词法分析

- 4 词法分析器的构造: [Educoder](#)、[校内](#) ([IP链接](#))



Lex程序

□ Lex程序包括三个部分

声明

%%

翻译规则

%%

辅助过程

□ Lex程序的翻译规则 (p_i 是模式)

p_1 {动作1}

p_2 {动作2}

.....

p_n {动作 n }

%{

/* tokens.h 包含常量 LT, LE, EQ, NE, GT, GE, WHILE, DO, ID, NUMBER, RELOP 的定义, 它们用 C 的 #define 方式来写。*/

#include "tokens.h"

int addSymbol();

int addNumber();

%}

/* 正规定义 */

delim [\t\n]

ws {delim}+

letter [A-Za-z]

digit [0-9]

id {letter}({letter}|{digit})*

number {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%{和%}包含的代码将直接复制到生成的分析器源程序文件中

对名字delim的引用

\.是转义序列, 表示点本身;
因为点是正规式中的元符号, 表示匹配换行符以外的任意单一字符



Lex程序

□ Lex程序包括三个部分

声明

%%

翻译规则

%%

辅助过程

5个字母连接
成的正规式

{ws}

while

do

{id}

{number}

对正规定义中名字ws的引用

/* 没有动作，也不返回 */

{return (WHILE);}

{return (DO);}

{yyval = installId (); return (ID);}

{yyval = installNum();

return (NUMBER);}

“ < ”

{yyval = LT; return (RELOP);}

“ <= ”

{yyval = LE; return (RELOP);}

“ = ”

{yyval = EQ; return (RELOP);}

“ <> ”

{yyval = NE; return (RELOP);}

“ > ”

{yyval = GT; return (RELOP);}

“ >= ”

{yyval = GE; return (RELOP);}

□ Lex程序的翻译规则 (p_i 是模式)

p_1 {动作1}

p_2 {动作2}

.....

p_n {动作 n }

- **yyval**: 定义在 lex.yy.c 中，它保存所返回记号的属性值
- **yytext**: 指向当前识别出的词法单元的起始字符
- **yylen**: 词法单元的长度



Lex程序

□ Lex程序包括三个部分

声明

%%

翻译规则

%%

辅助过程

□ Lex程序的翻译规则

(p_i 是模式)

p_1 {动作1}

p_2 {动作2}

.....

p_n {动作 n }

```
installId ( ) {
```

```
    /* 把词法单元装入符号表并返回指针。
```

```
    yytext指向该词法单元的第一个字符，
```

```
    yyleng给出的它的长度          */
```

```
    return addSymbol(yytext, yyleng);
```

```
}
```

```
installNum ( ) {
```

```
    /* 类似上面的过程，但词法单元不是标识符而是数 */
```

```
    return addNumber(atof(yytext));
```

```
}
```



利用Flex生成词法分析器

- 生成词法分析源码，`xxx.l`是词法描述文件

```
flex -i -I xxx.l
```

- 编译得到词法分析器可执行文件，`-ll`表示链接lex运行时库libl.a

```
gcc -g lex.yy.c -o xxx -ll
```

如果遇到找不到libl.a，可以将`-ll`改为`-lfl`，它代表去链接libfl.a 或者libfl.so

现代flex为`-lfl`



ANTLR 的文法文件 .g4的格式

□ 格式

```
grammar MyG;  
options { ... }  
import XXX;  
tokens { ... }  
@actionName { ... }  
ruleName : <stuff> ;  
.....
```

正规定义, DIGIT不是记号

模式定义, 词法状态

点通配任一字符

□ 纯词法

```
lexer grammar MyG;
```

□ ruleName

■ 词法: 大写 字母开头

■ 语法: 小写 字母开头

□ 词法规则

```
INT : DIGIT+ ;  
fragment DIGIT : [0-9] ;  
LQUOTE : '"' -> more, mode(STR) ;  
mode STR;  
STRING : '"' -> mode(DEFAULT_MODE);  
TEXT : . -> more
```

匹配此规则但继续寻找记号, 接下来匹配的记号规则包含本规则匹配的文本

模式调用



利用ANTLR生成词法分析器

□ exprLexer.g4

lexer grammar exprLexer;

```
tokens {  
    LeftParen,  
    RightParen,  
    Plus,  
    Minus,  
    Multiply,  
    Divide,  
    IntConst,  
}  
LeftParen: '(';  
RightParen: ')';  
  
Plus: '+' ;  
Minus: '-' ;  
Multiply: '*' ;  
Divide: '/' ;  
IntConst : [0-9]+ ;  
ID: [a-zA-Z_][a-zA-Z_0-9]* ;  
WS: [ \t\r\n]+ -> skip;
```

\$ antlr4 exprLexer.g4

\$ javac *.java

\$ grun exprLexer tokens -tokens <expr 程序 >



exprLexer.java

词法分析器的类定义

exprLexer.tokens

词法记号名及其数值

exprLexer.interp

包含允许运行内建解释器的数据，用于支撑IDEs对文法的调试

使用说明: <https://github.com/antlr/antlr4/blob/master/doc/index.md>

支持ANTLR4文法的VS Code扩展:

<https://marketplace.visualstudio.com/items?itemName=mike-lischke.vscode-antlr4>



ANTLR: Lexer规则中的命令

□ 命令格式

TokenName : 选项1|...|选项N [-> 命令名 [(参数)]];

□ 命令

- **skip**: 不返回记号给parser, 如识别出空白符或注释
- **more**: 取另一个记号但不抛出当前的文本
- **type(T)**: 设置当前记号的类型
- **channel(C)**: 设置当前记号的通道, 缺省为Token.DEFAULT_CHANNEL(值为0);
Token.HIDDEN_CHANNEL(值为1)
- **mode(M)**: 匹配当前记号后, 切换到模式M
- **pushMode(M)**: 与mode(M)类似, 但将当前模式入栈
- **popMode**: 从模式栈弹出模式, 使之成为当前模式



中国科学技术大学
University of Science and Technology of China

自上而下解析：一般方法



自上而下解析的一般方法

□ 自上而下 (top-down) 解析

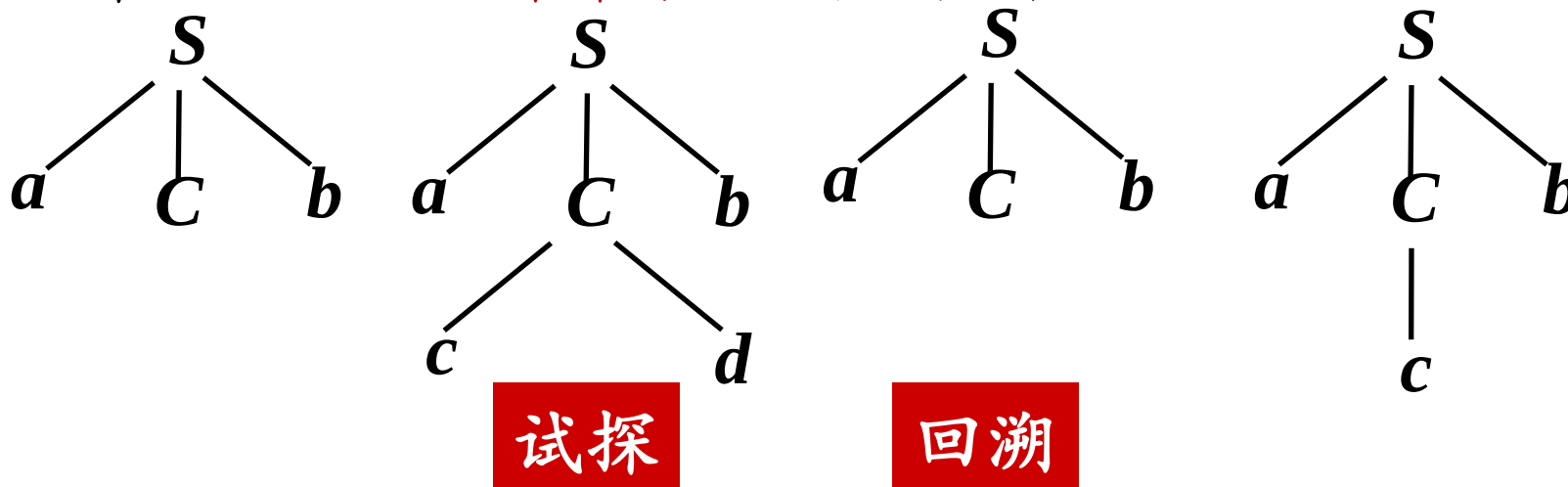
从开始符号出发, 为输入串寻找**最左推导**

试探 产生式的选择 – 失败**回溯**(效率低, 代价高)

■ ANTLR: 引入**带谓词的DFA**使得**回溯**后不重新分析输入串

例 文法 $S \rightarrow aCb$ $C \rightarrow cd \mid c$

为输入串 $w = acb$ **建立解析树** (从根结点开始)





自上而下解析：左递归

□ 文法左递归

- 若文法中存在某非终结符 A ，使得 $A \Rightarrow^+ A\alpha$ ，其中 α 是文法符号串

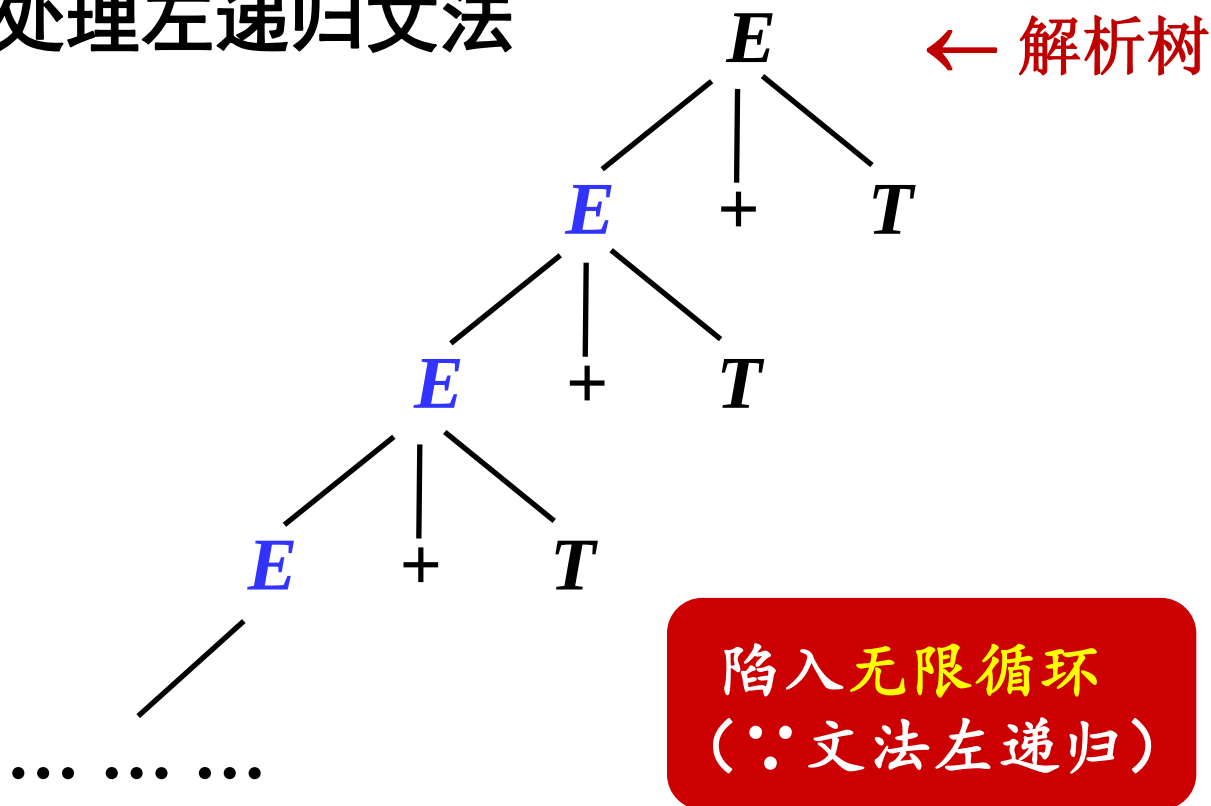
□ 自上而下解析不能处理左递归文法

算术表达式文法

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$





消除左递归(Eliminating left recursion)

□ 文法左递归

- 若文法中存在某非终结符 A , 使得 $A \Rightarrow^+ A\alpha$, 其中 α 是文法符号串

□ 直接左递归 (immediate left recursion)

若文法中存在 $A \rightarrow A\alpha \mid \beta$, 其中文法符号串 β 不以 A 开头

- 由 A 推出的串的特点 $A \Rightarrow^+ \beta\alpha \dots \alpha$

□ 消除直接左递归

文法 $A \rightarrow A\alpha \mid \beta$, 可写成 $A \rightarrow \beta A'$

$$A' \rightarrow \alpha A' \mid \varepsilon$$



消除左递归

例 算术表达式文法

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

消除左递归后的文法

$$E \rightarrow TE'$$

$$E' \rightarrow + TE' \mid \varepsilon$$

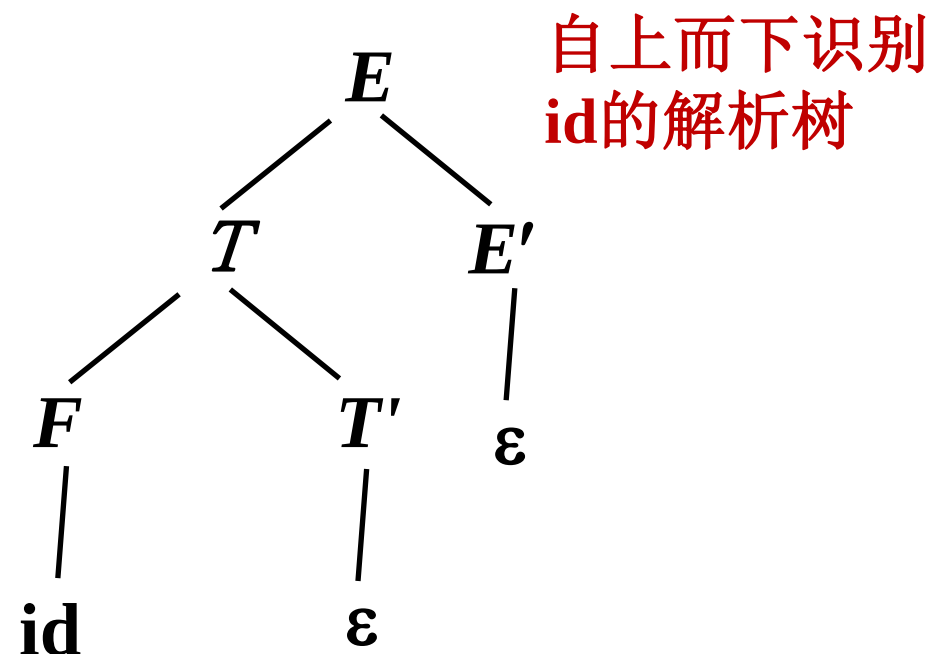
$$T \rightarrow FT'$$

$$T' \rightarrow * FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id}$$

$$E \Rightarrow^+ T + T \dots + T$$

$$T \Rightarrow^+ F * F \dots * F$$





消除非直接左递归

□ 间接左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow \textcolor{red}{S}d \mid \varepsilon$$

□ 先变换成直接左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow \textcolor{blue}{A}a\textcolor{blue}{d} \mid \textcolor{blue}{b}d \mid \varepsilon$$

□ 再消除左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow bd A' \mid A'$$

$$A' \rightarrow adA' \mid \varepsilon$$

□ 隐藏左递归

$$S \rightarrow AB\textcolor{red}{C}$$

$$\textcolor{red}{A} \rightarrow \varepsilon$$

$$B \rightarrow A\textcolor{red}{B}e \mid Cd$$

$$C \rightarrow S f$$

$B \Rightarrow ABe \Rightarrow Be$ 是隐藏直接左递归，
而 $S \Rightarrow ABc \Rightarrow Bc \Rightarrow Cdc \Rightarrow Sfdc$ 是
隐藏间接左递归。

ANTLR4 接受的文法可包含直接左递归，但不可包含间接或隐藏的左递归



提左因子(left factoring)

□ 有左因子的(left-factored)文法: $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

■ 自上而下解析时, 不清楚应该用A的哪个选择来代换

□ 提左因子

$$A \rightarrow \alpha A' \qquad A' \rightarrow \beta_1 \mid \beta_2$$

例 悬空else的文法

$$\begin{aligned} stmt \rightarrow & \text{if } expr \text{ then } stmt \text{ else } stmt \\ & \mid \text{if } expr \text{ then } stmt \mid \text{other} \end{aligned}$$

提左因子

$$\begin{aligned} stmt \rightarrow & \text{if } expr \text{ then } stmt \text{ optional_else_part} \mid \text{other} \\ \text{optional_else_part} \rightarrow & \text{else } stmt \mid \varepsilon \end{aligned}$$



LL(1)文法

L-scanning from left to right; **L**- leftmost derivation

□ 对文法加什么样的限制可以保证没有回溯？

□ 先定义两个和文法有关的函数

■ 符号串 α 的开始符号集合 $\text{FIRST}(\alpha)$

$$\text{FIRST}(\alpha) = \{a \mid \alpha \Rightarrow^* a..., a \in V_T\} \cup \{\varepsilon \mid \alpha \Rightarrow^* \varepsilon\}$$

↑ α 推出的非空串中首终结符和 α 推出空串时的 ε 组成的集合

■ A 的后继符号集合 $\text{FOLLOW}(A)$

$$\text{FOLLOW}(A) = \{a \mid S \Rightarrow^* ...Aa..., a \in V_T\} \cup \{\$ \mid S \Rightarrow^* ...A\}$$

↑ A 的后继符号集合：在句型中紧跟在 A 之后的终结符或 $\$$ 组成的集合

这里约定 $\$$ 为输入记号串的结束标志， S 是开始符号



LL(1)文法: FIRST(X)

□ 计算FIRST(X), $X \in V_T \cup V_N$

■ $X \in V_T$, $\text{FIRST}(X) = \{X\}$

■ $X \in V_N$ 且 $X \rightarrow Y_1 Y_2 \dots Y_k$

如果 $a \in \text{FIRST}(Y_i)$ 且 ε 在 $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$ 中, 则将 **a** 加入到 $\text{FIRST}(X)$

如果 ε 在 $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_k)$ 中, 则将 ε 加入到 $\text{FIRST}(X)$

■ $X \in V_N$ 且 $X \rightarrow \varepsilon$, 则将 ε 加入到 $\text{FIRST}(X)$

FIRST集合只可能包含终结符和 ε



LL(1)文法: FIRST

□ 计算 $\text{FIRST}(X_1 X_2 \dots X_n)$,

$X_i \in V_T \cup V_N$, 它包含

- $\text{FIRST}(\mathcal{E}_1)$ 中所有的非 ϵ 符号
- $\text{FIRST}(\mathcal{E}_i)$ 中所有的非 ϵ 符号,
如果 ϵ 在 $\text{FIRST}(\mathcal{E}_1)$, □,
 $\text{FIRST}(\mathcal{E}_{i-1})$ 中
- ϵ , 如果 ϵ 在 $\text{FIRST}(\mathcal{E}_1)$, □,
 $\text{FIRST}(\mathcal{E}_i)$ 中

FIRST集合只可能包含终结符和 ϵ

例 $E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(\emptyset) = \{ (, \text{id} \}$

$\text{FIRST}(E') = \{ +, \epsilon \}$

$\text{FIRST}(T') = \{ *, \epsilon \}$



LL(1)文法: FOLLOW

□ 计算 $\text{FOLLOW}(A)$, $A \in V_N$

- $\exists$ 加入到 $\text{FOLLOW}(\Sigma)$ 中
- 如果 $A \rightarrow \alpha B\beta$, 则将 $\text{FIRST}(\beta)$ 中所有终结符加入到 $\text{FOLLOW}(B)$
- 如果 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B\beta$ 且 $\epsilon \in \text{FIRST}(\beta)$, 则 $\text{FOLLOW}(A)$ 中的所有符号加入到 $\text{FOLLOW}(B)$

$$\text{FOLLOW}(A) \subseteq V_T \cup \{\$\}$$

例

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow + TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow * FT' \mid \epsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

$$\begin{aligned} \text{FIRST}(E) &= \text{FIRST}(T) = \text{FIRST}(\Phi) = \{ (, \text{id} \} \\ \text{FIRST}(E') &= \{ +, \epsilon \} \\ \text{FIRST}(T') &= \{ *, \epsilon \} \\ \text{FOLLOW}(E) &= \text{FOLLOW}(E') = \{), \exists \} \\ \text{FOLLOW}(T) &= \text{FOLLOW}(T') = \{ +,), \exists \} \\ \text{FOLLOW}(\Phi) &= \{ +, *,), \exists \} \end{aligned}$$



LL(1)文法

□ LL(1)文法的定义

任何两个产生式 $A \rightarrow \alpha \mid \beta$ 都满足下列条件:

■ $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$

■ 若 $\beta \Rightarrow^* \varepsilon$, 那么 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$

L-scanning from left to right 从左到右扫描输入

L- leftmost derivation 产生最左推导

1-决定分析器的每步动作时需要向前查看下一个输入符号
(即输入指针所指向的符号)

例 对于下面文法, 面临 $a...$ 时, 第2步推导不知用 A 的哪个产生式选择

$$S \rightarrow A B$$

$$A \rightarrow a b \mid \varepsilon$$

$$B \rightarrow a C$$

$$C \rightarrow \dots$$

$$\because a \in \text{FIRST}(ab) \cap \text{FOLLOW}(A)$$



LL(1)文法

□ LL(1)文法的定义

任何两个产生式 $A \rightarrow \alpha \mid \beta$ 都满足下列条件:

- $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$
- 若 $\beta \Rightarrow^* \varepsilon$, 那么 $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$

□ LL(1)文法有一些明显的性质

- 没有公共左因子
- 不是二义的
- 不含左递归



LL(k)文法

□ LL(1)文法的定义可以扩展到LL(k)

任何两个产生式 $A \rightarrow \alpha \mid \beta$ 都满足下列条件:

■ $\text{FIRST}(k, \alpha) \cap \text{FIRST}(k, \beta) = \emptyset$

■ 若 $\beta \Rightarrow^* \varepsilon$, 那么 $\text{FIRST}(k, \alpha) \cap \text{FOLLOW}(k, A) = \emptyset$

$\text{FIRST}(k, \alpha)$ 表示文法符号串 α 的前 k 个终结字符串集合
 $\text{FOLLOW}(k, A)$ 表示跟在 A 后的前 k 个终结字符串集合

L- scanning from left to right 从左到右扫描输入

L- leftmost derivation 产生最左推导

k - 决定解析器的每步动作时需要向前查看 k 个输入符号
(即输入指针所指向的符号)



中国科学技术大学
University of Science and Technology of China

ANTLR的使用



ANTLR 的语法文件 .g4

<https://github.com/antlr/antlr4/blob/master/doc/parser-rules.md>

□ 格式

```
grammar MyG;  
options { ... }  
import ... ;  
tokens { ... }  
channels {...} //lexer only  
@actionName { ... }  
ruleName : <stuff> ;  
.....
```

```
channels {  
    WHITESPACE_CHANNEL,  
    COMMENTS_CHANNEL  
}
```

模式定义
词法状态

□ ruleName

- 词法：大写 字母开头
- 语法：小写 字母开头

□ 纯词法分析器

```
lexer grammar MyG;
```

正规定义，
DIGIT不是记号

□ 词法规则

```
INT : DIGIT+ ;  
fragment DIGIT : [0-9] ;  
LQUOTE : '"' -> more, mode (STR) ;  
mode STR;  
STRING : '"' -> mode (DEFAULT_MODE);
```

模式调用

用在词法规则中，设置当前分析的通道，
可跳过空白符或注释

```
WS: [\t\n\r]+ -> channel (WHITESPACE);  
缺省的通道为 Token.DEFAULT_CHANNEL
```

<https://github.com/antlr/antlr4/blob/master/doc/lexer-rules.md>



ANTLR: Lexer命令

□ 命令格式

TokenName : <选项> -> 命令名 [(参数)]

□ 命令

- **skip**: 不返回记号给parser, 如识别出空白符或注释
- **type(T)**: 设置当前记号的类型
- **channel(C)**: 设置当前记号的通道, 缺省为Token.DEFAULT_CHANNEL(值为0);
Token.HIDDEN_CHANNEL(值为1)
- **mode(M)**: 匹配当前记号后, 切换到模式M
- **pushMode(M)**: 与mode(M)类似, 但将当前模式入栈
- **popMode**: 从模式栈弹出模式, 使之成为当前模式

<https://github.com/antlr/antlr4/blob/master/doc/lexer-rules.md>



词法语法切换

ANTLR lab, where you can learn about ANTLR or experiment with and test grammars! Just hit the **Run** button to try out the sample grammar.

To start developing with ANTLR, see [getting started](#).

[Feedback/issues](#) welcome. Brought to you by [Terence Parr](#), the maniac behind ANTLR.

Disclaimer: This website and related functionality are not meant to be used for private code, data, or other intellectual property. Assume everything you enter could become public! Grammars and input you enter are submitted to a unix box for execution and possibly persisted on disk or other mechanism. Please run antlr4-lab locally to avoid privacy concerns.

Lexer Parser Sample ?

```
1 parser grammar ExprParser;
2 options { tokenVocab=ExprLexer; }
3
4 program
5     : stat EOF
6     | def EOF
7     ;
8
9 stat: ID '=' expr ';'
10      | expr ';'
11      ;
12
13 def : ID '(' ID '(' ID* ')' '{' stat* '}'
14      ;
15
16 expr: ID
17      | INT
18      | func
19      | 'not' expr
20      | expr 'and' expr
21      | expr 'or' expr
22      ;
23 func : ID '(' expr '(' ID* ')' ')' ;
```

文法编写区

Input sample.expr ?

```
1 f(x,y) {
2   a = 3+foo;
3   x and y;
4 }
```

文法测试输入区

Start rule ?

program

Run

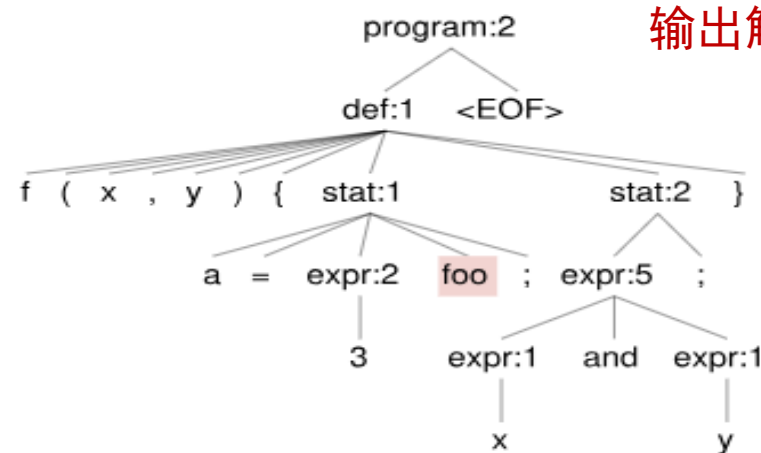
Show profiler

设置开始符号进行测试

Parser console

```
2:9 token recognition error at: '+'
2:10 extraneous input 'foo' expecting ';' ;
```

Tree Hierarchy





使用ANTLR构造解析器

□ 解析器的生成、编译和运行

```
$ antlr4 ExprParser.g4  
$ javac ExprParser*.java  
$ grun Expr program -gui sample.expr
```

```
antlr4='java org.antlr.v4.Tool'
```

```
grun='java org.antlr.v4.gui.TestRig'
```

-gui : 以图形界面 (GUI) 形式展示解析树。它会生成一个窗口，直观显示解析树结构。

-tree: 在终端以文本的层次结构表示解析树的树形结构，在没有图形界面环境时很有用。

-trace: 详细输出解析步骤，包括进入和退出每个规则时的状态，有助于调试和分析。

```
import org.antlr.v4.runtime.*;  
import java.nio.charset.Charset;  
import java.nio.file.Paths;  
import java.io.IOException;  
public class Main {  
    public static void main(String[] args) throws Exception {  
        Charset charset = Charset.defaultCharset ();  
        CharStream input = CharStreams.fromPath(Paths.get(args[0]), charset);  
        ExprLexer lexer = new ExprLexer(input);  
        CommonTokenStream tokens = new CommonTokenStream(lexer);  
        ExprParser parser = new ExprParser(tokens);  
        ParserRuleContext tree = parser.program(); // 开始解析  
        System.out.println(tree.toStringTree(parser)); // 打印语法树  
    }  
}
```



Parser规则

□ 语法规则

规则名：选择1 | ... | 选择n;

□ 规则的组成元素

- T、‘literal’：终结符、文本串 => 记号
- r：小写字母开头，代表非终结符
- r[参数]：传入一组逗号分隔的参数，相当于函数调用
- {动作}：在之前的元素之后、后继元素之前执行该动作
- {谓词}?：如果谓词为假，则不继续分析



Parser规则

□ 給规则选择加标签

- 格式： #标签名，后跟空格或换行

`e : e '*' e # Mult | e '+' e # Add | INT # Int ;`

ANTLR为每个标签产生规则上下文类，如 `XXXParser.MultContext`

□ 有何用处？

ANTLR会生成与该标签对应的语法结构的 `enter`和`exit`方法，
作为在遍历解析树时要监听触发的事件方法

```
public interface XXXListener extends ParseTreeListener {  
    void enterMult(XXXParser.MultContext ctx);  
    void exitMult(XXXParser.MultContext ctx);  
    .....  
}
```

`XXX`为用户设置的文法名称

<https://github.com/antlr/antlr4/blob/master/doc/parser-rules.md>



Parser规则

□ 給规则选择加标签

■ 可以复用标签

`e : e '*' e # BinaryOp | e '+' e # BinaryOp | INT # Int ;`

ANTLR为每个标签产生规则上下文类，如 `XXXParser.BinaryOpContext`

□ 有何用处？

ANTLR会生成与该标签对应的语法结构的 **enter**和**exit**方法

```
public interface XXXListener extends ParseTreeListener {  
    void enterBinaryOp (XXXParser. BinaryOpContext ctx);  
    void exitBinaryOp (XXXParser. BinaryOpContext ctx);  
    .....  
}
```

XXX为用户设置的文法名称

<https://github.com/antlr/antlr4/blob/master/doc/parser-rules.md>



Parser规则

□ 规则属性定义

规则名[参数args] returns [返回值rvals] locals [局部变量lvars] : ... ;

例如:

```
add[int x] returns [int ret] : '+' INT { $ret = $x + $INT.int; } ;
```

□ 带内嵌动作的规则: 可以引用上下文对象

```
grammar Expr;
```

```
@header {  
package tools;  
import java.util.*;  
}
```

Java代码, 直接复制到生成
的解析器源码的头部

```
@parser::members {  
/** "memory" for our calculator; variable/value pairs go here */  
Map<String, Integer> memory = new HashMap<String, Integer>();  
int eval(int left, int op, int right) {  
    switch ( op ) {  
        case MUL : return left * right;  
        case ADD : return left + right;  
        .....  
    }  
    return 0;  
}  
}
```

自定义解析器类的成员
✓ 数据成员memory
✓ 方法成员eval



Parser规则

□ 带内嵌动作的规则：可以引用上下文对象

```
program: stat+ ;
stat:   e NEWLINE      {System.out.println($e.v);}
      | ID '=' e NEWLINE {memory.put($ID.text, $e.v); System.out.println($ID.text+"="+$e.v);}
      | NEWLINE
      ;
e returns [int v] : a=e op=('*'|'/') b=e {$v = eval($a.v, $op.type, $b.v);}
  | a=e op=('+'|'-') b=e {$v = eval($a.v, $op.type, $b.v);}
  | INT                {$v = $INT.int;}
  | ID { String id = $ID.text; $v = memory.containsKey(id) ? memory.get(id) : 0; }
  | '(' e ')'          {$v = $e.v;}
  ;
MUL : '*' ;
DIV : '/' ;
ADD : '+' ;
SUB : '-' ;
ID  : [a-zA-Z]+ ; // match identifiers
INT : [0-9]+ ;    // match integers
NEWLINE: '\r'? '\n' ; // return newlines to parser (is end-statement signal)
WS : [ \t]+ -> skip ; // toss out whitespace
```

输入程序及
其解析结果

```
a=3 +4 ;
b=a+4 ;
```

```
a=7
b=11
(program (stat a = (e (e 3) + (e 4)) ; \n) (stat b = (e (e a) + (e 4)) ;
```



Grammar Imports

□ 将语法分解成多个可复用的块

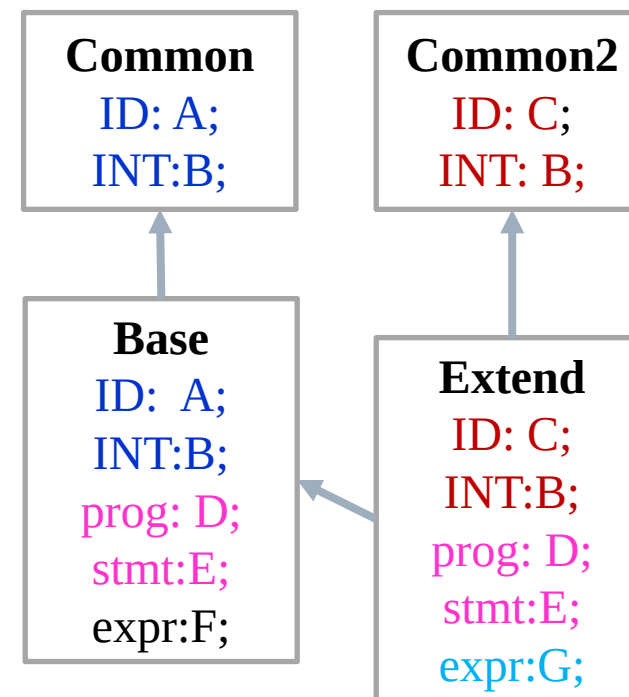
■ 通过import导入的语法类似于面向对象语言中的超类

```
lexer grammar Common;  
ID : [a-zA-Z_][a-zA-Z_0-9]* ;  
INT : [0-9]+ ;  
WS : [ \t\r\n]+ -> skip ;
```

```
grammar Base;  
import Common;  
prog : stmt* ;  
stmt : expr ';' ;  
expr : ID ;
```

```
lexer grammar Common2;  
ID : [a-zA-Z][a-zA-Z_0-9]* ;  
INT : [0-9]+ ;  
WS : [ \t\r\n]+ -> skip ;
```

```
grammar Extend;  
import Common2,Base;  
  
// 扩展表达式规则  
expr : ID | INT ;
```



<https://github.com/antlr/antlr4/blob/master/doc/grammars.md>



思考题

- ANTLR4 容许哪些类型的左递归？
- ANTLR4 对所支持的左递归如何处理？例如，对下面两种情况分别会怎样解析？

Exp : Exp '*' Exp | Exp '+' Exp | IntConst;

Exp : Exp '+' Exp | Exp '*' Exp | IntConst;

- ANTLR 能为上面哪种情况构造出符号‘*’的优先级比‘+’高的表达式解析器？这是基于 ANTLR 采用的何种二义性消除规则？
- 如果将下面的第1行改写成第2行，那么生成的解析器源码有什么样的变化？请理解和说明 '# Mult' 的作用和意义。

Exp : Exp '*' Exp | Exp '+' Exp | IntConst;

Exp : Exp '*' Exp # Mult | Exp '+' Exp # Add | IntConst # Int ;

- 给出ANTLR 不支持的左递归文法的例子并分析原因



自下而上解析

(移进-归约解析)

- 归约(最右推导的逆过程)
- 句柄(可归约串),可能不唯一
- 冲突: 移进-归约、归约-归约



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$
 $A \rightarrow Abc \mid b$
 $B \rightarrow d$

输入串: $abbcde$

一步归约：将与某个产生式右部匹配
的子串代换成该产生式的左部符号



如 $A \rightarrow b$



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

输入串: $abbcde$

ab (读入 ab) 寻找能匹配某产生式右部的子串

$A \rightarrow b$

$$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$$

右句型: 按最右推导推出的句型, $aABe$ 、 $aAde$ 、 $aAbcde$ 、 $abbcde$ 都是右句型



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

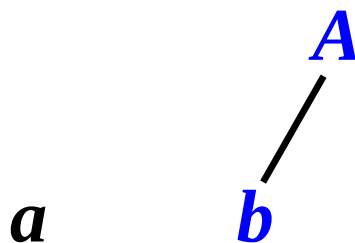
例 $S \rightarrow aABe$
 $A \rightarrow Abc \mid b$
 $B \rightarrow d$

输入串: $abbcde$

ab

aA (归约)

用产生式 $A \rightarrow b$
归约后仍能形成右句型
 $aAbcde$ 是右句型



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$

右句型: 按最右推导推出的句型, $aABe$ 、 $aAde$ 、 $aAbcde$ 、 $abbcde$ 都是右句型



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

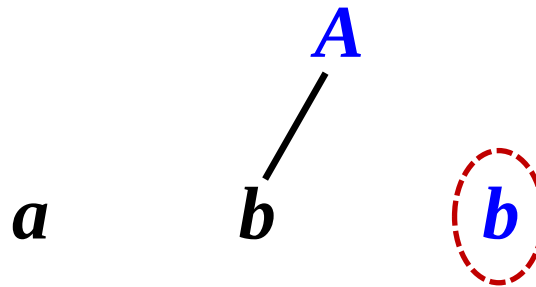
例 $S \rightarrow aABe$
 $A \rightarrow Abc \mid b$
 $B \rightarrow d$

输入串: $abbcde$

ab

aAb (再读入 b)

$A \rightarrow b$
 b 可以归约成 A 吗?



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



归约 (Reduce)

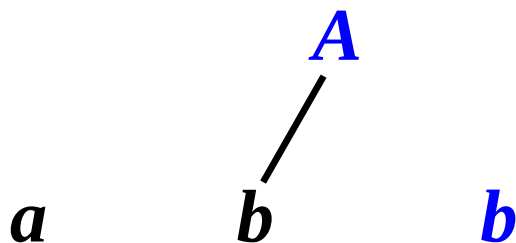
把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$
 $A \rightarrow Abc \mid b$
 $B \rightarrow d$

输入串: $abbcde$

ab

aAb (再读入 b)



$A \rightarrow b$
 b 可以归约成 A 吗?

归约后能否形成右句型?
 $aAAcde$ 不是右句型
故不能将 b 归约成 A

$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$

$A \rightarrow \textcolor{blue}{Abc} \mid b$

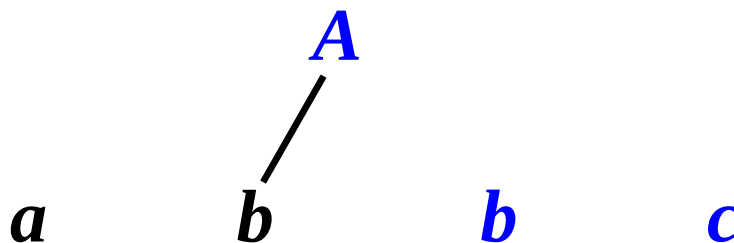
$B \rightarrow d$

$A \rightarrow \textcolor{blue}{Abc}$

输入串: $abbcde$

ab

$a\textcolor{blue}{Abc}$ (再读入 c)



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} a\textcolor{blue}{Abc}de \Rightarrow_{rm} abbcde$



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$

$A \rightarrow \textcolor{blue}{Abc} \mid b$

$B \rightarrow d$

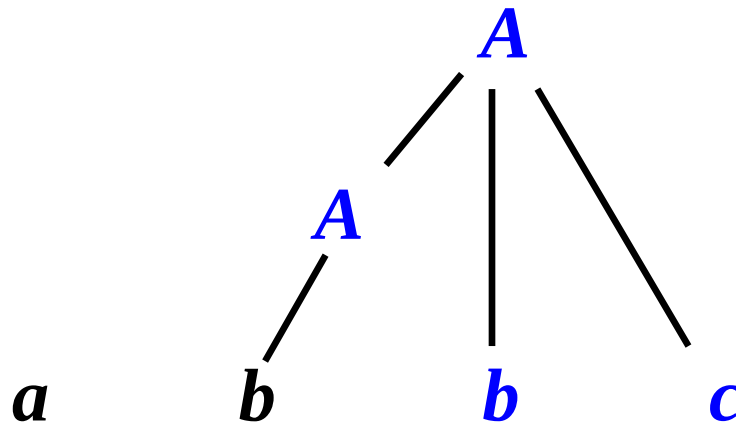
$A \rightarrow \textcolor{blue}{Abc}$

输入串: $abbcde$

ab

$a\textcolor{blue}{Abc}$

$a\textcolor{blue}{A}$ (归约)



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} a\textcolor{blue}{A}de \Rightarrow_{rm} a\textcolor{blue}{Abc}de \Rightarrow_{rm} abbcde$



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

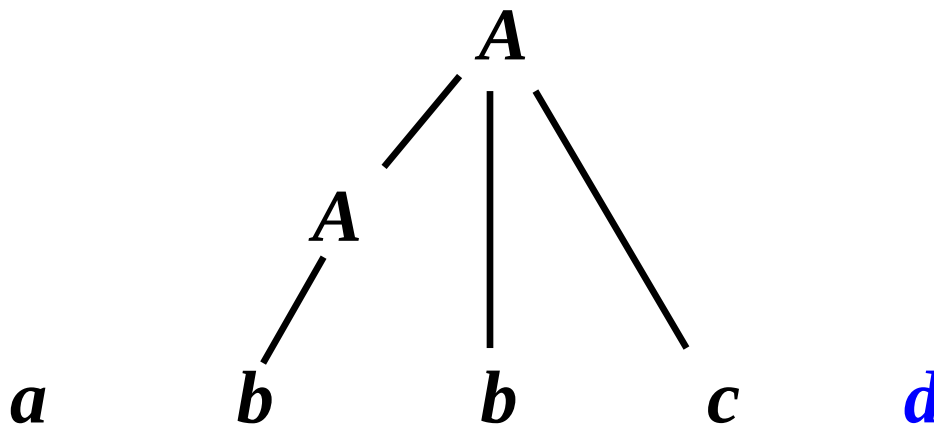
$B \rightarrow d$

输入串: $abbcde$

ab

$aAbc$

aAd (再读入 d)



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

$B \rightarrow d$

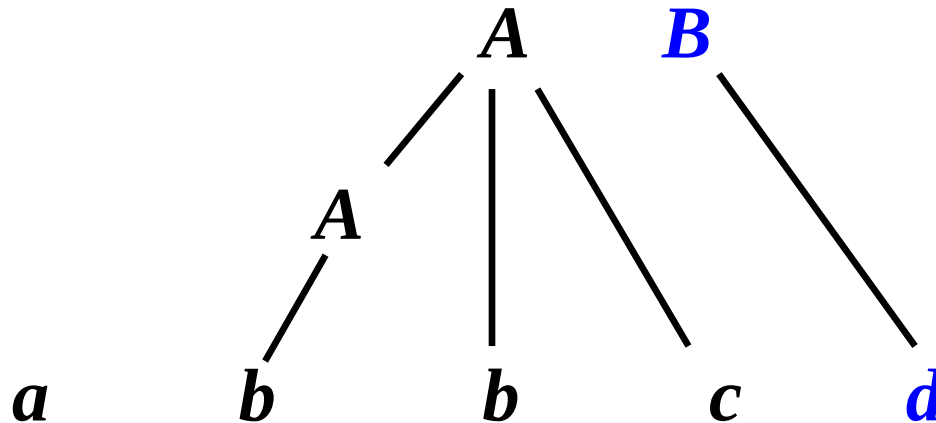
输入串: $abbcde$

ab

$aAbc$

aAd

aAB (归约)



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

输入串: $abbcd e$

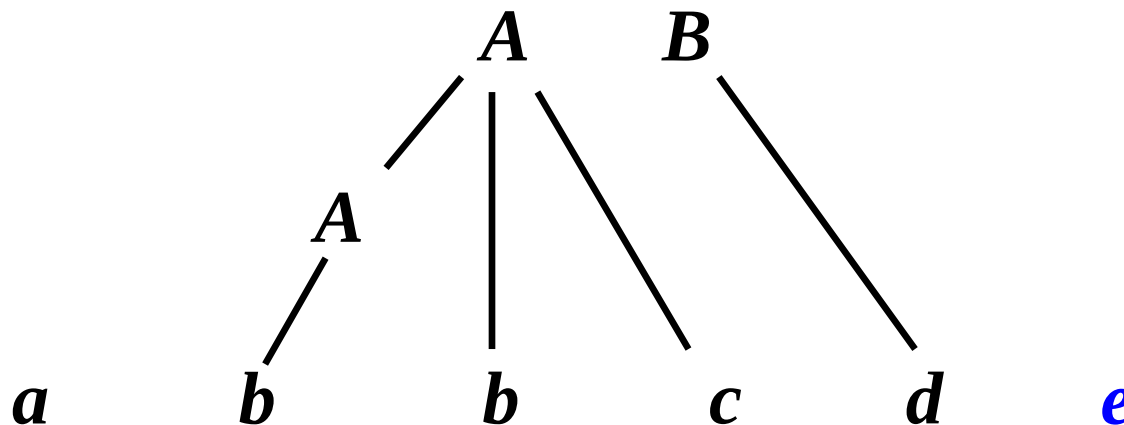
a b

a A b c

a A d

a A B

a A B e (再读入 e)



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} a\underline{Abc}de \Rightarrow_{rm} abbcde$



归约 (Reduce)

把输入串归约成文法的开始符号，是最右推导的逆过程

例 $S \rightarrow aABe$

$A \rightarrow Abc \mid b$

$B \rightarrow d$

输入串: $abbcd e$

ab

$aAbc$

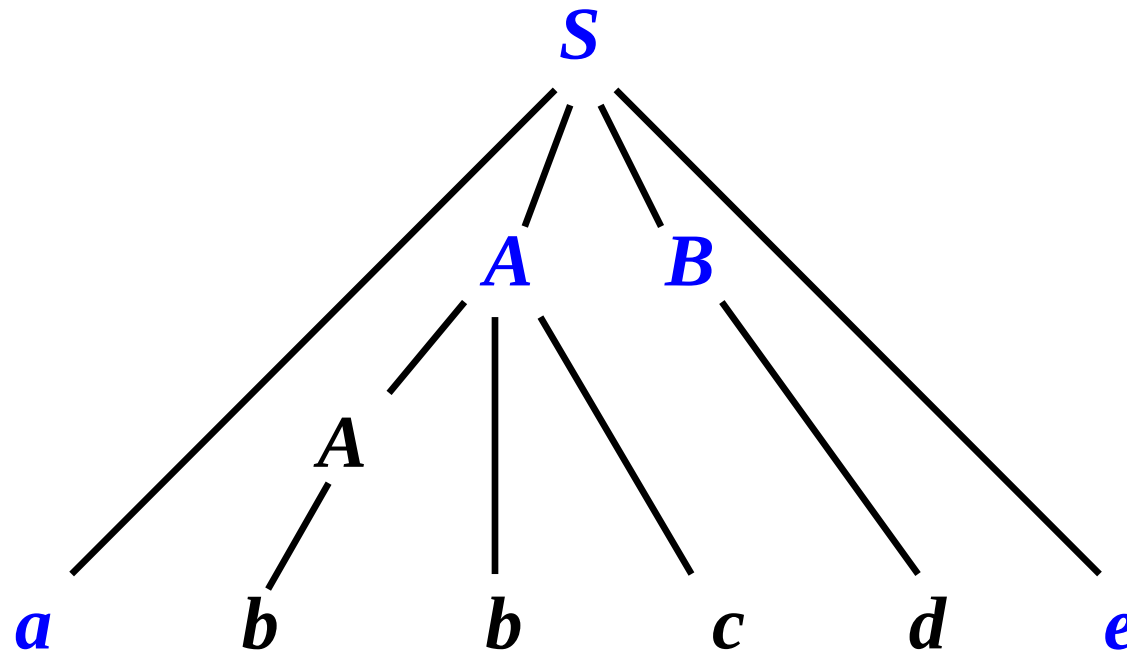
aAd

aAB

$aABe$

S (归约)

$S \rightarrow aABe$



$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$



自下而上解析

(移进-归约解析)

- 归约(最右推导的逆过程)
- 句柄(可归约串),可能不唯一
- 冲突: 移进-归约、归约-归约



句柄(handles)

□ 右句型的句柄（可归约串）

■ 是右句型 $\alpha\beta w$ 中和某产生式 $A \rightarrow \beta$ 右部匹配的最左子串 δ , 并且

■ 把 β 归约成 A 代表了最右推导的逆过程的一步

右句型 $\alpha\beta w$ 中将子串 β 归约成 A 后得到的串 αAw 仍是右句型

$S \rightarrow aABe$

* 右句型：最右推导可得的句型

$A \rightarrow Abc \mid b$

$B \rightarrow d$

$S \Rightarrow_{rm} aABe \Rightarrow_{rm} aAde \Rightarrow_{rm} aAbcde \Rightarrow_{rm} abbcde$

■ 句柄的右边仅含终结符（是尚未处理输入串）

■ 如果文法二义，那么句柄可能不唯一



例句柄不唯一

$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

$$E \Rightarrow_{rm} E * E$$

$$\Rightarrow_{rm} E * E + E$$

$$\Rightarrow_{rm} E * E + \text{id}_3$$

$$\Rightarrow_{rm} E * \text{id}_2 + \text{id}_3$$

$$\Rightarrow_{rm} \text{id}_1 * \text{id}_2 + \text{id}_3$$

$$E \Rightarrow_{rm} E + E$$

$$\Rightarrow_{rm} E + \text{id}_3$$

$$\Rightarrow_{rm} E * E + \text{id}_3$$

$$\Rightarrow_{rm} E * \text{id}_2 + \text{id}_3$$

$$\Rightarrow_{rm} \text{id}_1 * \text{id}_2 + \text{id}_3$$



例 句柄不唯一

$$E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

$$E \Rightarrow_{rm} E * E$$

$$\Rightarrow_{rm} E * E + E$$

$$\Rightarrow_{rm} E * E + \text{id}_3$$

$$\Rightarrow_{rm} E * \text{id}_2 + \text{id}_3$$

$$\Rightarrow_{rm} \text{id}_1 * \text{id}_2 + \text{id}_3$$

$$E \Rightarrow_{rm} E + E$$

$$\Rightarrow_{rm} E + \text{id}_3$$

$$\Rightarrow_{rm} E * E + \text{id}_3$$

$$\Rightarrow_{rm} E * \text{id}_2 + \text{id}_3$$

$$\Rightarrow_{rm} \text{id}_1 * \text{id}_2 + \text{id}_3$$

在右句型 $E * E + \text{id}_3$ 中，句柄不唯一： id_3 、 $E * E$

* 右句型：最右推导可得句型



中国科学技术大学
University of Science and Technology of China

用栈实现移进-归约解析



用栈实现移进-归约解析

先通过“移进-归约解析器在解析输入串 $id_1 * id_2 + id_3$ 时的动作序列”来了解移进-归约解析的工作方式。

需要引入**栈**保存移进的文法符号

归约时需要从栈的顶部将形成句柄的文法符号串弹出，再将归约成的非终结符入栈

初始格局：

栈	输入
\$	w\$

解析成功后的格局：

栈	输入
\$S	\$



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3$ \$	



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3$ \$	移进



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
\$ <i>E</i>	$* id_2 + id_3 \$$	



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
$\$E$	$* id_2 + id_3 \$$	移进



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
$\$E$	$* id_2 + id_3 \$$	移进
$\$E*$	$id_2 + id_3 \$$	



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
SE	$* id_2 + id_3 \$$	移进
$SE*$	$id_2 + id_3 \$$	移进



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
SE	$* id_2 + id_3 \$$	移进
$SE*$	$id_2 + id_3 \$$	移进
$SE*id_2$	$+ id_3 \$$	



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
$\$E$	$* id_2 + id_3 \$$	移进
$\$E*$	$id_2 + id_3 \$$	移进
$\$E*id_2$	$+ id_3 \$$	按 $E \rightarrow id$ 归约



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
\$ E	$* id_2 + id_3 \$$	移进
\$ $E*$	$id_2 + id_3 \$$	移进
\$ $E*id_2$	$+ id_3 \$$	按 $E \rightarrow id$ 归约
\$ $E*E$	$+ id_3 \$$	



移进-归约解析: $id_1 * id_2 + id_3$

栈	输 入	动 作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
\$ E	$* id_2 + id_3 \$$	移进
\$ $E*$	$id_2 + id_3 \$$	移进
\$ $E*id_2$	$+ id_3 \$$	按 $E \rightarrow id$ 归约
\$ $E*E$	$+ id_3 \$$	移进? 归约?
$\begin{aligned} E &\Rightarrow_{rm} E * E \\ &\Rightarrow_{rm} E * E + E \\ &\Rightarrow_{rm} E * E + id_3 \end{aligned}$ 移进 $\begin{aligned} E &\Rightarrow_{rm} E + E \\ &\Rightarrow_{rm} E + id_3 \\ &\Rightarrow_{rm} E * E + id_3 \end{aligned}$ 归约		



移进-归约解析: $id_1 * id_2 + id_3$

栈	输入	动作
\$	$id_1 * id_2 + id_3 \$$	移进
\$ id_1	$* id_2 + id_3 \$$	按 $E \rightarrow id$ 归约
\$E	$* id_2 + id_3 \$$	移进
\$E*	$id_2 + id_3 \$$	移进
\$ E* id_2	$+ id_3 \$$	按 $E \rightarrow id$ 归约
\$ E*E	$+ id_3 \\$	移进
\$ E*E+	$id_3 \$$	移进
\$ E*E+ id_3	\$	按 $E \rightarrow id$ 归约
\$ E*E+E	\$	按 $E \rightarrow E+E$ 归约
\$ E*E	\$	按 $E \rightarrow E*E$ 归约
\$E	\$	接受



自下而上解析

(移进-归约解析)

- 归约(最右推导的逆过程)
- 句柄(可归约串),可能不唯一
- 冲突: 移进-归约、归约-归约



移进-归约解析需要解决的一些问题

- 怎么确定右句型中要归约的子串(即句柄)
 - 句柄总是出现在栈顶 (长度大于等于0的语法符号串)
- 面临**移进-归约冲突**时, 如何决策是选择移进还是归约?
- 面临**归约-归约冲突**时, 不知该选择哪一个产生式归约?
 - 栈顶形成不同的句柄, 或
 - 句柄是不止一个产生式的右部



移进-归约解析的冲突

□ 移进-归约冲突(shift/reduce conflict)

例

$stmt \rightarrow$ **if** $expr$ **then** $stmt$
 | **if** $expr$ **then** $stmt$ **else** $stmt$
 | **other**

如果移进-归约解析器处于格局(configuration)

栈

输入

... **if** $expr$ **then** $stmt$

else ... \$

一般地，优先移进
也满足else的
就近匹配原则



移进-归约解析的冲突

□ 归约-归约冲突(reduce/reduce conflict)

$stmt \rightarrow id (parameter_list) \mid expr = expr$ $id(...)$ 是过程调用

$parameter_list \rightarrow parameter_list, parameter \mid parameter$

$parameter \rightarrow id$

$expr \rightarrow id (expr_list) \mid id$ $id(...)$ 也表示数组元素的引用

$expr_list \rightarrow expr_list, expr \mid expr$

由 $A(I, J)$ 开始的语句

栈

... id (id

归约成 $expr$ 还是 $parameter$?

输入

, id)...

方法1
一般用位于前面的产生式进行归约



移进-归约解析的冲突

□ 归约-归约冲突(reduce/reduce conflict)

$stmt \rightarrow \text{id}(parameter_list) \mid expr = expr$

$parameter_list \rightarrow parameter_list, parameter \mid parameter$

$parameter \rightarrow id$

$expr \rightarrow id(expr_list) \mid id$ id(...)也表示数组元素的引用

$expr_list \rightarrow expr_list, expr \mid expr$

由 $A(I, J)$ 开始的语句(词法分析查符号表, 区分第一个id)

栈

... **procid**(id

输入

, id)...

■ 需要修改上面的文法

方法2
改写文法, 区分
id是否是procid

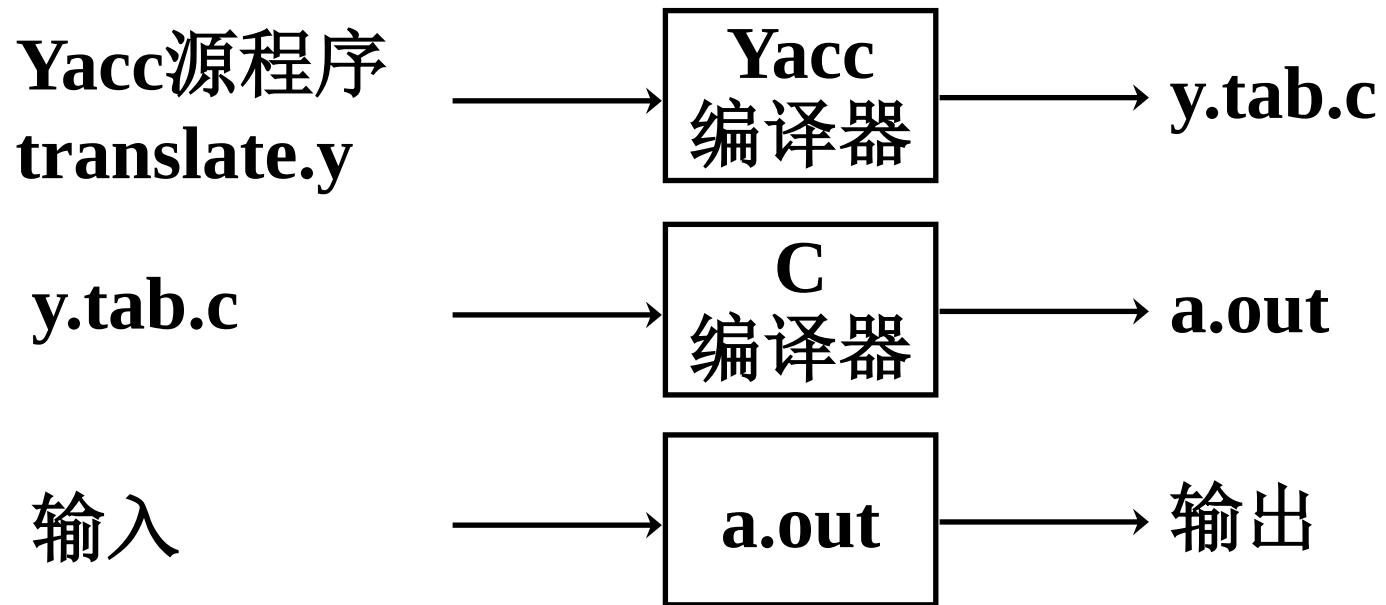


中国科学技术大学
University of Science and Technology of China

自下而上解析器的生成器：YACC



□ YACC (Yet Another Compiler Compiler)





例 简单计算器

- 输入一个表达式并回车，显示计算结果
- 也可以输入一个空白行

声明部分

```
%{  
# include <ctype .h>  
# include <stdio.h >  
# define YYSTYPE double /*将栈定义为double类型 */  
%}
```

```
%token NUMBER  
%left '+' '-'  
%left '*' '/'  
%right UMINUS  
%%
```



例 简单计算器

翻译规则部分

```
lines      : lines expr '\n'    {printf ( "%g \n", $2 ) }  
           | lines '\n'  
           | /*  $\epsilon$  */  
           ;  
  
expr       : expr '+' expr      { $$ = $1 + $3; }  
           | expr '-' expr      { $$ = $1 - $3; }  
           | expr '*' expr      { $$ = $1 * $3; }  
           | expr '/' expr      { $$ = $1 / $3; }  
           | '(' expr ')'       { $$ = $2; }  
           | '-' expr %prec UMINUS { $$ = -$2; }  
           | NUMBER  
           ;
```

%%



例 简单计算器

翻译规则部分

```
lines      : lines expr '\n'    {printf ( "%g \n", $2 ) }  
           | lines '\n'  
           | /*  $\epsilon$  */  
           ;  
  
expr       : expr '+' expr      { $$ = $1 + $3; }  
           | expr '-' expr      { $$ = $1 - $3; }  
           | expr '*' expr      { $$ = $1 * $3; }  
           | expr '/' expr      { $$ = $1 / $3; }  
           | '(' expr ')'       { $$ = $2; }  
           | '-' expr %prec UMINUS { $$ = -$2; }  
           | NUMBER  
           ;
```

%%

-5+10看成是-(5+10), 还是(-5)+10? 取后者



例 简单计算器

C例程部分

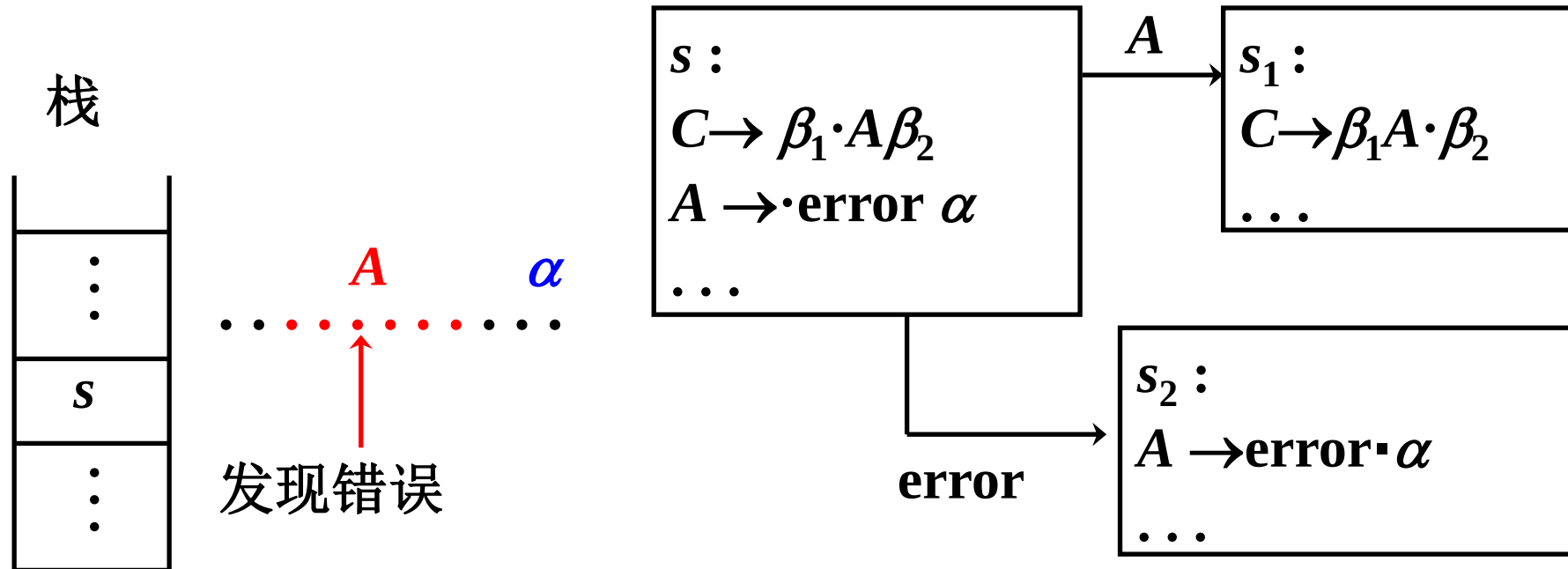
```
yylex ( ) {  
    int c;  
    while ( ( c = getchar ( ) ) == ' ' );  
    if ( ( c == '.' ) || (isdigit (c) ) ) {  
        ungetc (c, stdin);  
        scanf ( "% lf ", &yylval);  
        return NUMBER;  
    }  
    return c;  
}
```

为了C编译器能准确报告yylex函数中错误的位置，
需要在生成的程序y.tab.c中使用编译命令#line



YACC的错误恢复

- 增加错误产生式 $A \rightarrow \text{error } \alpha$
- 遇到语法错误时

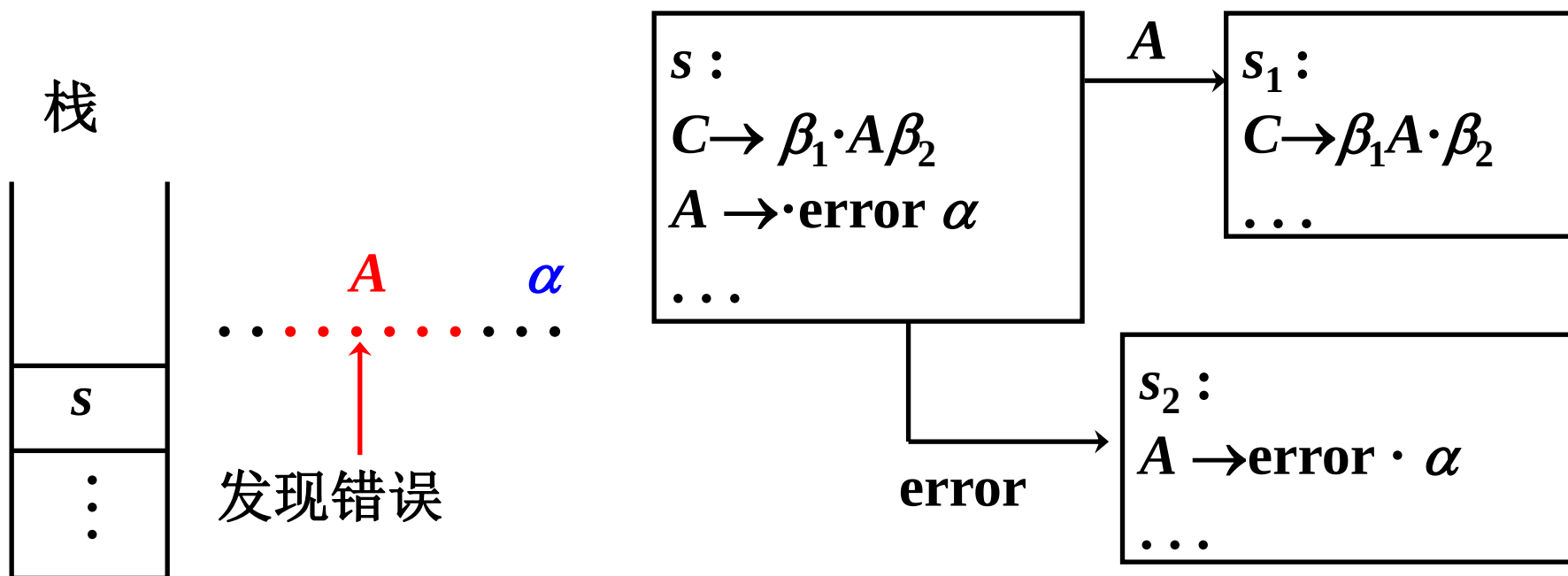




YACC的错误恢复

□ 遇到语法错误时

- 从栈中弹出状态，直到发现栈顶状态的项目集包含形为 $A \rightarrow \cdot \text{error } \alpha$ 的项目为止

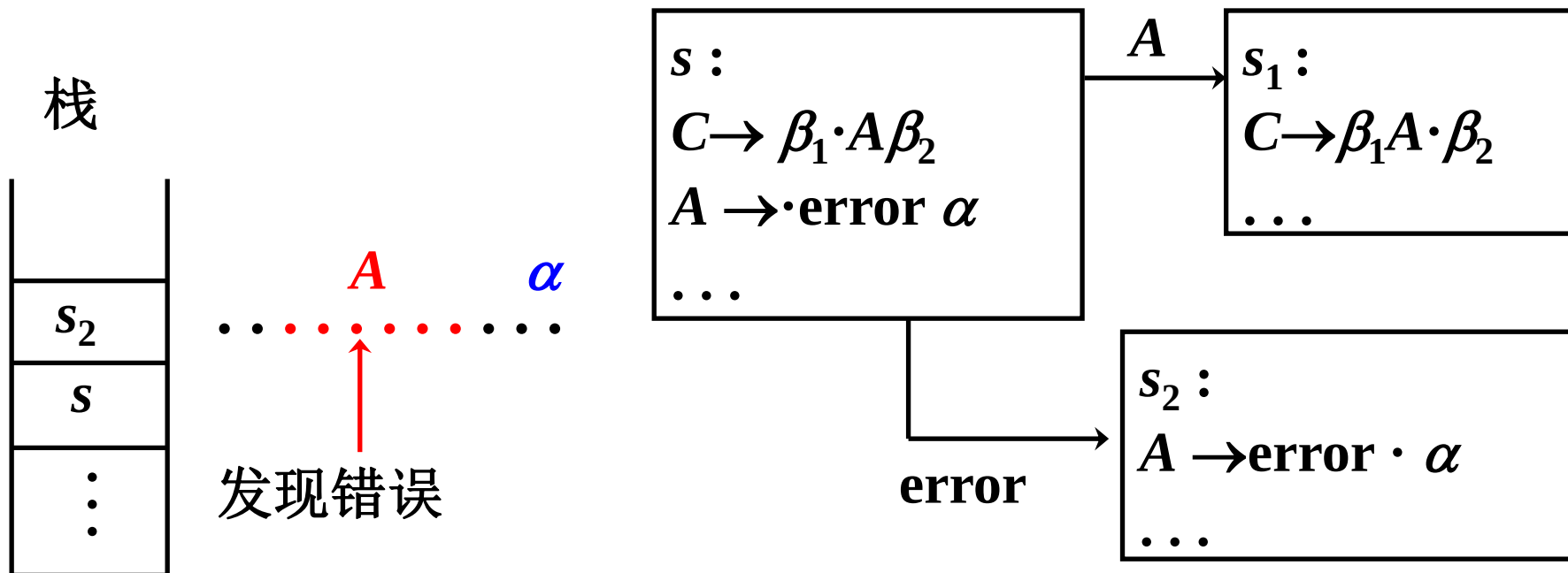




YACC的错误恢复

□ 遇到语法错误时

- 从栈中弹出状态，直到发现栈顶状态的项目集包含形为 $A \rightarrow \cdot \text{error } \alpha$ 的项目为止
- 把虚构的终结符 **error** “移进” 栈

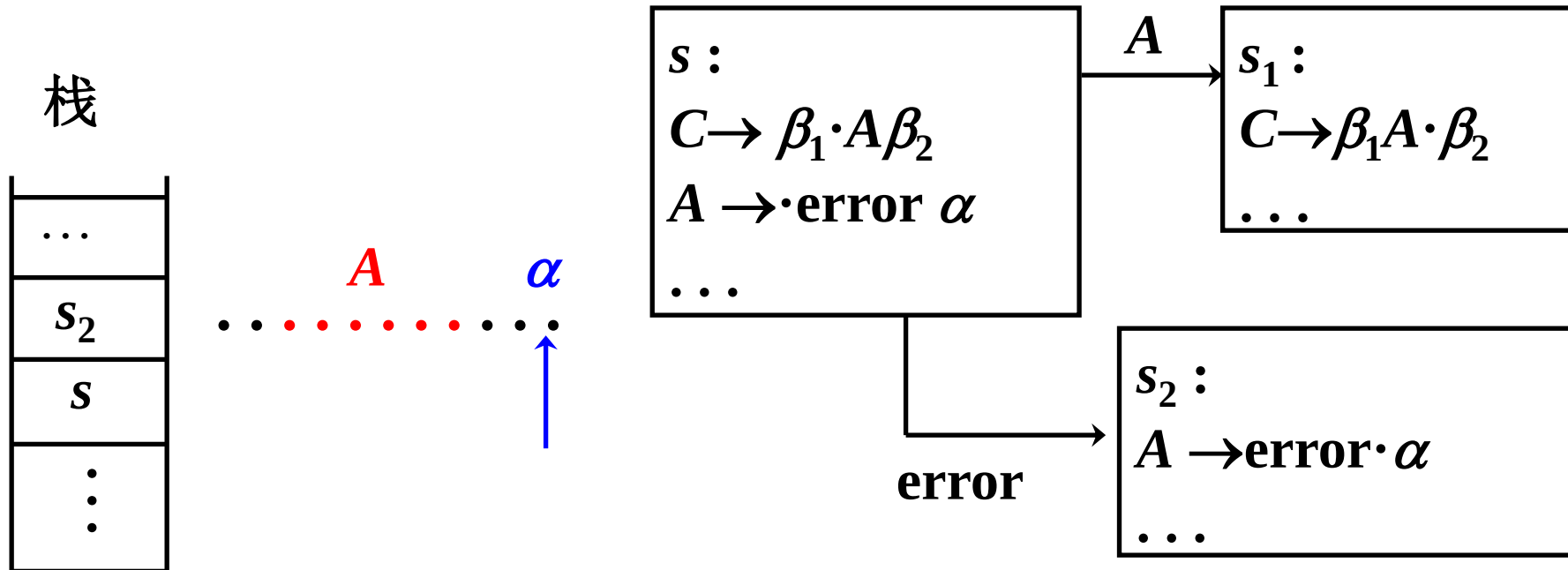




YACC的错误恢复

□ 遇到语法错误时

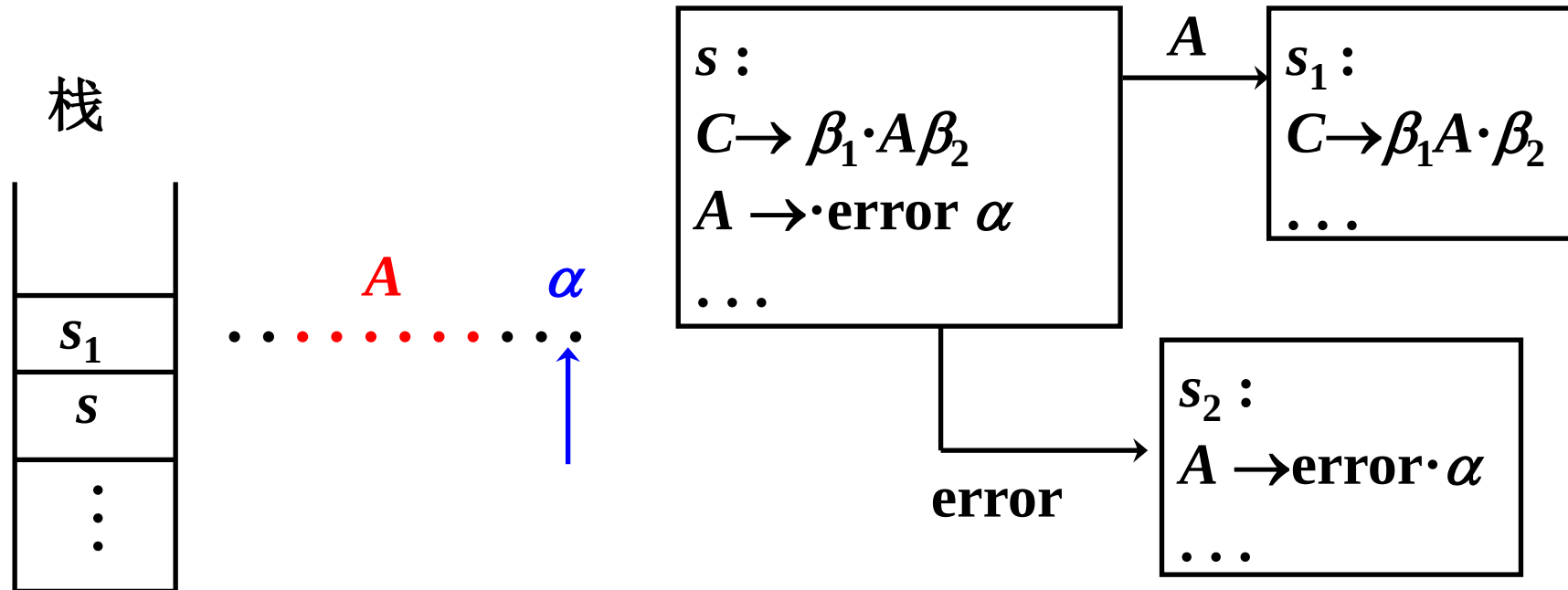
- 从栈中弹出状态，直到发现栈顶状态的项目集包含形为 $A \rightarrow \cdot \text{error } \alpha$ 的项目为止
- 把虚构的终结符error “移进” 栈
- 忽略若干输入符号，直至找到 α ，把 α 移进栈





YACC的错误恢复

- 从栈中弹出状态，直到发现栈顶状态的项目集包含形为 $A \rightarrow \cdot \text{error } \alpha$ 的项目为止
- 把虚构的终结符 **error** “移进” 栈
- 忽略若干输入符号，直至找到 α ，把 α 移进栈
- 把 **error** α 归约为 A ，恢复正常分析





例 简单计算器

□ 增加错误恢复的简单计算器

```
lines      : lines expr '\n'      {printf ( "%g \n", $2 ) }  
           | lines '\n'  
           | /*  $\epsilon$  */  
           | error '\n'{yyerror ( “重新输入上一行” );  
                               yyerrok;}  
           ;
```